Program Studi Teknik Elektro ITB

Nama Kuliah (Kode) : Praktikum Arsitektur Sistem Komputer (EL3111)
Tahun / Semester : 2023-2024 / Ganjil
Modul : 1 (Instruction Fetch dan Instruction Decode)
Nama Asisten / NIM : _____
Nama Praktikan / NIM : Danny Rahmat Pratama / 13224019

Tugas Pendahuluan

Soal

1. **Rancang dan verifikasikan komponen multiplexer 2-to-1 generik selebar 32-bit yang memilih keluaran D dari dua masukan A dan B berdasarkan sinyal selektor sel (0→A, 1→B). Desain harus synthesizable dan diuji dengan simulasi fungsional serta simulasi timing. Cantumkan kode dan tangkapan layar waveform dari kedua simulasi.**
   Jawab:

   Tabel 1-1 Kode Multiplexer 2 to 1 (32bit)

```
// Praktikum EL3011 Arsitektur Sistem Komputer
// Modul : 1
// Percobaan : 1
// Tanggal : 21 Oktober 2026
// Nama (NIM) 1 : Danny Rahmat Pratama (13224019)
// Nama (NIM) 2 : Aziiz Adityatama (13224070)
// Nama File : multiplexer2_to_1.v
// Deskripsi : Implementasi Multiplexer 2:1 berukuran 32 bit dalam
Verilog
module multiplexer2_to_1(
    input [31:0] A,
    input [31:0] B,
    input sel,
    output [31:0] D
);

assign D = (sel == 1'b0) ? A : B;

endmodule
```

   Tabel 1-2 Kode Testbench Multiplexer 2 to 1 (32 bit)

```
// Praktikum EL3011 Arsitektur Sistem Komputer
// Modul : 1
// Percobaan : 1
// Tanggal : 21 Oktober 2026
// Nama (NIM) 1 : Danny Rahmat Pratama (13224019)
// Nama (NIM) 2 : Aziiz Adityatama (13224070)
// Nama File : tb_multiplexer2_to_1.v
// Deskripsi : Testbench untuk Multiplexer 2:1 berukuran 32 bit dalam
```

```
Verilog
module tb_multiplexer2_to_1;

// Variable for testing
reg [31:0]  A; // Driving the A input
reg [31:0]  B; // Driving the B input
reg         sel; // Driving the  selector input
wire [31:0]  D; // Drving the D output
integer     error; // Integer for detecting errors
integer     i; // Integer for testing loop

// Initiate the DUT from the multiplexer module
multiplexer2_to_1 u0 (
    .A(A),
    .B(B),
    .sel(sel),
    .D(D)
);

// Initialization for variables to zero
task init;
begin
    A <= 0;
    B <= 0;
    sel <= 0;
    error <= 0;
end
endtask

// Checking for each test
task check;
begin
    if ((sel == 1'b0) && (D !== A)) begin

$display("------------------------------------------------------------");
        $display("T= %0t | Output tidak sesuai dengan selektor yang
dipilih", $time);
        $display("T= %0t | A = %d | B = %d | sel = %b | D = %d ",
$time, A, B, sel, D);
        error = error + 1;
    end else if ((sel == 1'b1) && (D != B)) begin

$display("------------------------------------------------------------");
        $display("T= %0t | Output tidak sesuai dengan selektor yang
dipilih", $time);
        $display("T= %0t | A = %d | B = %d | sel = %b | D = %d ",
$time, A, B, sel, D);
```

```
        error = error + 1;
    end else begin

$display("------------------------------------------------------------");
        $display("T= %0t | Output sesuai dengan selektor yang
dipilih", $time);
        $display("T= %0t | A = %d | B = %d | sel = %b | D = %d ",
$time, A, B, sel, D);
    end
end
endtask

// Testing for random inputs
task testing;
begin
    for(i = 0; i < 10; i = i + 1) begin
        #10
        A <= $random;
        B <= $random;
        sel <= $random;
        check;
    end
end
endtask

initial begin
    // Dumping variables for GTKWave
    $dumpfile("multiplexer2_to_1.vcd");
    $dumpvars(0, tb_multiplexer2_to_1);

    // Testbenches
    init;
    check; // Initial checking for zeros
    testing; // Testing the mux

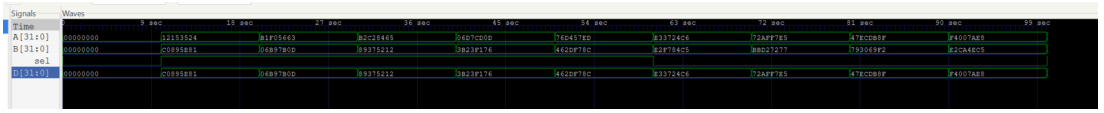
    // Results
    $display("Testing ended with %d erros", error);
    $finish;
end

endmodule
```

Tabel 1-3 Hasil Testbench Multiplexer 2 to 1 (32 bit)

```
VCD info: dumpfile multiplexer2_to_1.vcd opened for output.
------------------------------------------------------------
T= 0 | Output sesuai dengan selektor yang dipilih
T= 0 | A =       x | B =       x | sel = x | D =       x
------------------------------------------------------------
T= 10 | Output sesuai dengan selektor yang dipilih
```

```
T= 10 | A =         0 | B =         0 | sel = 0 | D =         0
----------------------------------------------------------
T= 20 | Output sesuai dengan selektor yang dipilih
T= 20 | A =  303379748 | B = 3230228097 | sel = 1 | D = 3230228097
----------------------------------------------------------
T= 30 | Output sesuai dengan selektor yang dipilih
T= 30 | A = 2985317987 | B =  112818957 | sel = 1 | D =  112818957
----------------------------------------------------------
T= 40 | Output sesuai dengan selektor yang dipilih
T= 40 | A = 2999092325 | B = 2302104082 | sel = 1 | D = 2302104082
----------------------------------------------------------
T= 50 | Output sesuai dengan selektor yang dipilih
T= 50 | A =  114806029 | B =  992211318 | sel = 1 | D =  992211318
----------------------------------------------------------
T= 60 | Output sesuai dengan selektor yang dipilih
T= 60 | A = 1993627629 | B = 1177417612 | sel = 1 | D = 1177417612
----------------------------------------------------------
T= 70 | Output sesuai dengan selektor yang dipilih
T= 70 | A = 3812041926 | B = 3807872197 | sel = 0 | D = 3812041926
----------------------------------------------------------
T= 80 | Output sesuai dengan selektor yang dipilih
T= 80 | A = 1924134885 | B = 3151131255 | sel = 0 | D = 1924134885
----------------------------------------------------------
T= 90 | Output sesuai dengan selektor yang dipilih
T= 90 | A = 1206705039 | B = 2033215986 | sel = 0 | D = 1206705039
----------------------------------------------------------
T= 100 | Output sesuai dengan selektor yang dipilih
T= 100 | A = 4093672168 | B = 3804909253 | sel = 0 | D = 4093672168
Testing ended with         0 erros
```



Gambar 1-1 Hasil Waveform Functional Analysis Multiplexer 2 to 1 (32 bit)

	Clock	Slack	End Point TNS
1	virtual_clk	1.085	0.000

Gambar 1-2 Hasil Timing Analysis untuk Setup Report

	Clock	Slack	End Point TNS
1	virtual_clk	5.656	0.000

Gambar 1-3 Hasil Timing Analysis untuk Hold Report

2. **Rancang dan verifikasikan komponen 4-adder selebar 32-bit yang menghitung PC_4_add = PC_old + 4. Simpanlah desain dengan nama pc_4_adder_rv32i.v, kemudian desain harus synthesizable dan diuji dengan simulasi fungsional serta simulasi timing. Cantumkan kode dan tangkapan layar waveform dari kedua simulasi.**

   Jawab:

Tabel 2-1 Kode 4-Adder

```
// Praktikum EL3011 Arsitektur Sistem Komputer
// Modul : 1
// Percobaan : 1
// Tanggal : 21 Oktober 2026
// Nama (NIM) 1 : Danny Rahmat Pratama (13224019)
// Nama (NIM) 2 : Aziiz Adityatama (13224070)
// Nama File : pc_4_adder_rv32i.v
// Deskripsi : Implementasi 4-Adder Program Counter RISC-V di Verilog
module pc_4_adder_rv32i(
    input   [31:0] PC_old,
    output  [31:0] PC_4_add
);

assign PC_4_add = PC_old + 4;

endmodule
```

Tabel 2-2 Kode Testbench 4-Adder

```
// Praktikum EL3011 Arsitektur Sistem Komputer
// Modul : 1
// Percobaan : 1
// Tanggal : 21 Oktober 2026
// Nama (NIM) 1 : Danny Rahmat Pratama (13224019)
// Nama (NIM) 2 : Aziiz Adityatama (13224070)
// Nama File : tb_pc_4_adder_rv32i.v
// Deskripsi : Testbench untuk 4-Adder Program Counter RISC-V di
Verilog
module tb_pc_4_adder_rv32i;

reg     [31:0] PC_old;
wire    [31:0] PC_4_add;
integer i;
integer error;

pc_4_adder_rv32i u0 (
    .PC_old(PC_old),
    .PC_4_add(PC_4_add)
);

task init;
begin
    PC_old <= 0;
    error = 0;
end
endtask
```

```
task check;
begin
    if(PC_4_add !== PC_old + 4) begin
        $display("-----------------------------------");
        $display("T=%0t | Output tidak sesuai!", $time);
        $display("T=%0t | PC_old = %d | PC_4_add = %d", $time, PC_old,
PC_4_add);
        error = error + 1;
    end else begin
        $display("-----------------------------------");
        $display("T=%0t | Output sesuai!", $time);
        $display("T=%0t | PC_old = %d | PC_4_add = %d", $time,
PC_old, PC_4_add);
    end
end
endtask

task testing;
begin
    for (i = 0; i < 10; i = i + 1) begin
        #10
        PC_old <= $random;
        check;
    end
end
endtask

initial begin
    // Dumpfile for gtkwave
    $dumpfile("pc_4_adder_rv32i.vcd");
    $dumpvars(0, tb_pc_4_adder_rv32i);

    // Testbench checking
    init;
    check;
    testing;

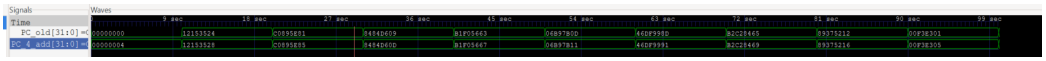
    // Finishing
    $display("Testbench ended with %d error", error);
    $finish;

end


endmodule
```

Tabel 2-3 Hasil Testbench 4-Adder

```
-----------------------------------
T=0 | Output sesuai!
T=0 | PC_old =         x | PC_4_add =         x
-----------------------------------
T=10 | Output sesuai!
T=10 | PC_old =         0 | PC_4_add =         4
-----------------------------------
T=20 | Output sesuai!
T=20 | PC_old =  303379748 | PC_4_add =  303379752
-----------------------------------
T=30 | Output sesuai!
T=30 | PC_old = 3230228097 | PC_4_add = 3230228101
-----------------------------------
T=40 | Output sesuai!
T=40 | PC_old = 2223298057 | PC_4_add = 2223298061
-----------------------------------
T=50 | Output sesuai!
T=50 | PC_old = 2985317987 | PC_4_add = 2985317991
-----------------------------------
T=60 | Output sesuai!
T=60 | PC_old =  112818957 | PC_4_add =  112818961
-----------------------------------
T=70 | Output sesuai!
T=70 | PC_old = 1189058957 | PC_4_add = 1189058961
-----------------------------------
T=80 | Output sesuai!
T=80 | PC_old = 2999092325 | PC_4_add = 2999092329
-----------------------------------
T=90 | Output sesuai!
T=90 | PC_old = 2302104082 | PC_4_add = 2302104086
-----------------------------------
T=100 | Output sesuai!
T=100 | PC_old =   15983361 | PC_4_add =   15983365
Testbench ended with          0 error
```



Gambar 2-1 Hasil Waveform Functional Analysis

	Clock	Slack	End Point TNS
1	virtual_clk	1.119	0.000

Gambar 2-2 Hasil Waveform Timing Analysis Setup Report

	Clock	Slack	End Point TNS
1	virtual_clk	5.201	0.000

Gambar 2-3 Hasil Waveform Timing Analysis Hold Report

3. **Buatlah komponen program counter (PC) selebar 32-bit untuk arsitektur single-cycle RV32I. PC merupakan register yang akan mengeluarkan dan menahan input berupa nilai PC selanjutnya setiap clock rising, kemudian mengeluarkan nilai 32'h0 saat sinyal reset active. Simpanlah**

desain dengan nama pc_rv32i.v, kemudian desain harus synthesizable dan diuji dengan simulasi fungsional serta simulasi timing. Cantumkan kode dan tangkapan layar waveform dari kedua simulasi.

Jawab:

Tabel 3-1 Kode Program Counter

```
// Praktikum EL3011 Arsitektur Sistem Komputer
// Modul : 1
// Percobaan : 1
// Tanggal : 28 Oktober 2026
// Nama (NIM) 1 : Danny Rahmat Pratama (13224019)
// Nama (NIM) 2 : Aziiz Adityatama (13224070)
// Nama File : pc_rv32i.v
// Deskripsi : Implementasi Program Counter 32 bit RISC-V pada Verilog
module pc_rv32i(
    input       [31:0]  PC_in,
    input               clk,
    input               reset,
    output  reg [31:0]  PC_out
);

always @(posedge clk or posedge reset) begin
    if(reset) begin
        PC_out <= 32'h0;
    end else begin
        PC_out <= PC_in;
    end
end

endmodule
```

Tabel 3-2 Kode Testbench Program Counter

```
// Praktikum EL3011 Arsitektur Sistem Komputer
// Modul : 1
// Percobaan : 1
// Tanggal : 28 Oktober 2026
// Nama (NIM) 1 : Danny Rahmat Pratama (13224019)
// Nama (NIM) 2 : Aziiz Adityatama (13224070)
// Nama File : tb_pc_rv32i.v
// Deskripsi : Testbench untuk Program Counter 32 bit RISC-V pada
Verilog
module tb_pc_rv32i;

reg     [31:0]  PC_in;
reg             reset;
reg             clk;
wire    [31:0]  PC_out;
```

```
integer i;
integer error;
reg     [2:0]   delay;

// Clock generation
always #10 clk = ~clk;
pc_rv32i u0 (
    .PC_in(PC_in),
    .clk(clk),
    .reset(reset),
    .PC_out(PC_out)
);

task init;
begin
    PC_in <= 0;
    reset <= 0;
    clk <= 0;
    error = 0;
end
endtask

task testing;
begin
    for(i = 0; i < 10; i = i + 1) begin
        delay = $random;
        #(delay)
        PC_in <= $random;
        reset <= $random;
    end
end
endtask

initial begin
    $dumpfile("pc_rv32i.vcd");
    $dumpvars(0, tb_pc_rv32i);

    init;
    testing;

    #10
    $display("Program selesai dengan %d error", error);
    $finish;
end

initial begin
    $monitor("------------------------------------------ \n T=%0t |
Terdapat Perubahan Input! \n T=%0t | clk = %d | PC_in = %d | reset =
%d | PC_out %d", $time, $time, clk, PC_in, reset, PC_out);
```

```
end

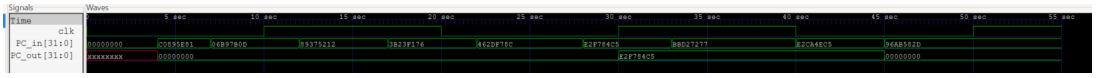
always @(posedge clk or posedge reset) begin
    #1
    if(reset) begin
        if(PC_out !== 32'h0) begin
            $display("-----------------------------------------");
            $display("T=%0t | Output tidak sesuai!", $time);
            $display("T=%0t | clk = %d | PC_in = %d | reset = %d |
PC_out %d", $time,clk, PC_in, reset, PC_out);
            error = error + 1;
        end else begin
            $display("-----------------------------------------");
            $display("T=%0t | Output sesuai!", $time);
            $display("T=%0t | clk = %d | PC_in = %d | reset = %d |
PC_out %d", $time,clk, PC_in, reset, PC_out);
        end
    end else begin
        if(PC_out !== PC_in) begin
            $display("-----------------------------------------");
            $display("T=%0t | Output tidak sesuai!", $time);
            $display("T=%0t | clk = %d | PC_in = %d | reset = %d |
PC_out %d", $time,clk, PC_in, reset, PC_out);
            error = error + 1;
        end else begin
            $display("-----------------------------------------");
            $display("T=%0t | Output sesuai!", $time);
            $display("T=%0t | clk = %d | PC_in = %d | reset = %d |
PC_out %d", $time,clk, PC_in, reset, PC_out);
        end
    end
end

endmodule
```

Tabel 3-3 Hasil Testbench Program Counter

```
-----------------------------------------
 T=0 | Terdapat Perubahan Input!
 T=0 | clk = 0 | PC_in =         0 | reset = 0 | PC_out        x
-----------------------------------------
 T=4 | Terdapat Perubahan Input!
 T=4 | clk = 0 | PC_in = 3230228097 | reset = 1 | PC_out        0
-----------------------------------------
T=5 | Output sesuai!
T=5 | clk = 0 | PC_in = 3230228097 | reset = 1 | PC_out        0
-----------------------------------------
 T=7 | Terdapat Perubahan Input!
 T=7 | clk = 0 | PC_in =  112818957 | reset = 1 | PC_out        0
```

```
--------------------------------------------
 T=10 | Terdapat Perubahan Input!
 T=10 | clk = 1 | PC_in =  112818957 | reset = 1 | PC_out        0
--------------------------------------------
T=11 | Output sesuai!
T=11 | clk = 1 | PC_in =  112818957 | reset = 1 | PC_out        0
--------------------------------------------
 T=12 | Terdapat Perubahan Input!
 T=12 | clk = 1 | PC_in = 2302104082 | reset = 1 | PC_out        0
--------------------------------------------
 T=17 | Terdapat Perubahan Input!
 T=17 | clk = 1 | PC_in =  992211318 | reset = 1 | PC_out        0
--------------------------------------------
 T=20 | Terdapat Perubahan Input!
 T=20 | clk = 0 | PC_in =  992211318 | reset = 1 | PC_out        0
--------------------------------------------
 T=22 | Terdapat Perubahan Input!
 T=22 | clk = 0 | PC_in = 1177417612 | reset = 1 | PC_out        0
--------------------------------------------
 T=28 | Terdapat Perubahan Input!
 T=28 | clk = 0 | PC_in = 3807872197 | reset = 0 | PC_out        0
--------------------------------------------
 T=30 | Terdapat Perubahan Input!
 T=30 | clk = 1 | PC_in = 3807872197 | reset = 0 | PC_out 3807872197
--------------------------------------------
T=31 | Output sesuai!
T=31 | clk = 1 | PC_in = 3807872197 | reset = 0 | PC_out 3807872197
--------------------------------------------
 T=33 | Terdapat Perubahan Input!
 T=33 | clk = 1 | PC_in = 3151131255 | reset = 0 | PC_out 3807872197
--------------------------------------------
 T=40 | Terdapat Perubahan Input!
 T=40 | clk = 0 | PC_in = 3804909253 | reset = 0 | PC_out 3807872197
--------------------------------------------
 T=45 | Terdapat Perubahan Input!
 T=45 | clk = 0 | PC_in = 2527811629 | reset = 1 | PC_out        0
--------------------------------------------
T=46 | Output sesuai!
T=46 | clk = 0 | PC_in = 2527811629 | reset = 1 | PC_out        0
--------------------------------------------
 T=50 | Terdapat Perubahan Input!
 T=50 | clk = 1 | PC_in = 2527811629 | reset = 1 | PC_out        0
--------------------------------------------
T=51 | Output sesuai!
T=51 | clk = 1 | PC_in = 2527811629 | reset = 1 | PC_out        0
Program selesai dengan          0 error
```



Gambar 3-1 Hasil Waveform Function Analysis

Slow 1200mV 85C Model

	Clock	Slack	End Point TNS
1	clk	0.506	0.000

Gambar 3-2 Hasil Timing Analysis Setup Report

Slow 1200mV 85C Model

	Clock	Slack	End Point TNS
1	clk	1.361	0.000

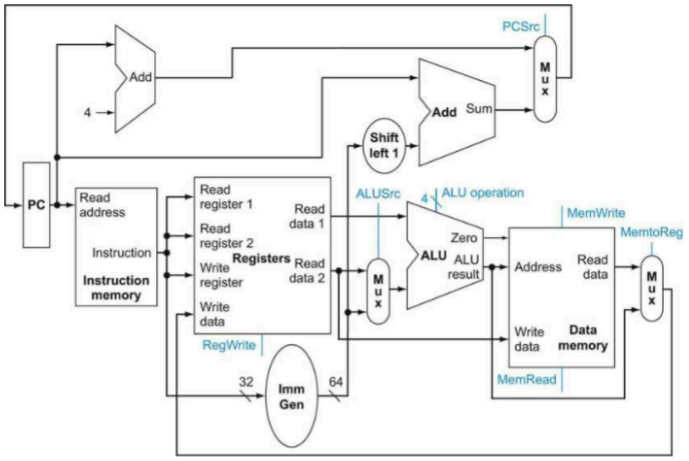
Gambar 3-3 Hasil Timing Analysis Hold Report

4. **Jelaskan alur eksekusi instruksi pada prosesor single-cycle: IF, ID, EX, MEM, WB (meski pada single cycle semua diselesaikan dalam 1 siklus)! Jelaskan semua format instruksi RV32I dan arti setiap field bit-nya!**

   Jawab:

   Pada praktikum ini mikroprosesor yang akan diuji adalah RISC-V sehingga eksekusi instruksi adalah berdasarkan mikroprosesor RISC-V. Terdapat lima tahap yang dilakukan ketika mikroprosesor melakukan eksekusi instruksi, yaitu:

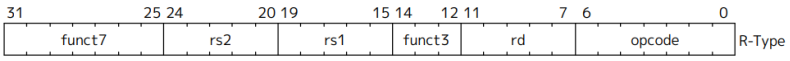
   1. Instruction Fetch (IF)
      Tahap IF berfungsi mengatur aliran instruksi yang akan diolah pada tahap berikutnya. Instruksi yang sedang dijalankan merupakan instruksi yang berasal dan disimpan dari memory. Pada tahap ini akan diambil 32-bit instruksi dari memori dan melakukan increment terhadap PC sebesar 4 untuk menunjuk pada instruksi setelahnya (pengambilan address memori per byte).
   2. Instruction Decode (ID)
      Pada tahap ini, instruksi dengan lebar 32-bit akan dipecah sesuai format instruksi yang digunakan. Secara umum, instruksi akan dipecah menjadi bagian alamat destinasi (rd), alamat register sumber (rs1 dan rs2), dan opcode atau kode yang menandakan operasi yang akan dijalankan.
   3. Execute / Address Calculation (EX)
      Pada tahap ini sebagian besar operasi aritmatika dan logika dilakukan pada Arithmetic and Logical Unit (ALU), misalnya penjumlahan untuk instruksi add. Pada tahap ini juga terdapat tempat untuk meneruskan alamat register kembali ke tahap ID sebagai deteksi hazard untuk pipeline implementation.
   4. Data Memory (MEM)
      Pada tahap ini, data disimpan dari data memory. Data memory hanya dapat disimpan atau dibaca jika sinyal MemRead dan/atau MemWrite yang sesuai sehingga operasi baca dan/atau tulis pada data memory dapat dilakukan.
   5. Write Back (WB)
      Tahap ini digunakan untuk mengalirkan data dari data memory atau hasil memory atau hasil perhitungan ALU ke register untuk disimpan dan dipakai untuk instruksi-instruksi berikutnya.



Gambar 4-1 Arsitektur Prosesor RISC-V Single-Cycle

Pada RISC-V RV32I, lebar dari instruksi adalah 32-bit dengan total terdapat enam tipe, yaitu:

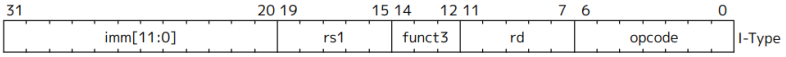
1. R-type



Gambar 4-2 Set Instruksi R-type

Format R-type digunakan untuk melakukan integer register-register instructions dimana register rs1 dan rs2 adalah sumber dan hasil dari aritmatika disimpan pada register rd. Operasi aritmatika yang dapat dilakukan adalah ADD, SLT, AND, OR, XOR, SLL, SRL, SUB, dan SRA.

2. I-type

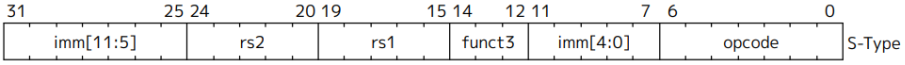


Gambar 4-3 Set Instruksi I-type

Format I-type digunakan untuk integer register-immediate instructions yang dimana register rs1 adalah sumber dan dilakukan operasi dengan 12-bit sign-extended immediate yang kemudian disimpan pada register rd. Operasi yang dapat dilakukan adalah ADDI, SLTI, ANDI, ORI, XORI, SLLI, SRLI, SRAI, LUI, dan AUIPC.

Selain itu, format I-type juga dapat dilakukan untuk menerima data dari memory. Hal ini dilakukan dengan melakukan pencarian address memori dari penjumlahan rs1 dengan signed-extend 12-bit offset. Nilai dari memory address tersebut kemudian disimpan pada register rd.

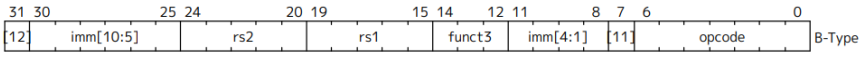
3. S-type



Gambar 4-4 Set Instruksi S-type

S-type digunakan untuk menyimpan data dari register menuju memory. Address Memory yang ingin dilakukan operasi dicari dengan menambahkan register rs1 dengan sign-extend 12-bit offset. S-type disini menyimpan nilai dari register rs2 menuju memory.

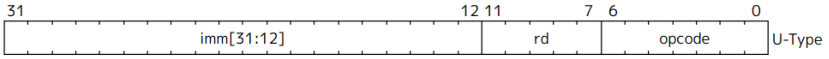
4. B-type



Gambar 4-5 Set Instruksi B-type

B-type digunakan untuk conditional branches. Nilai dari 12-bit B-immediate adalah signed-offset yang merupakan perkalian dari 2 byte. Nilai dari offset tersebut dilakukan sign-extend dan dijumlahkan terhadap address dari branch instruction untuk mendapatkan target address. Range dari conditional branch adalah +- 4KiB

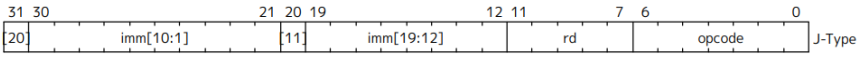
5. U-type



Gambar 4-6 Set Instruksi U-type

U-type digunakan untuk instruksi upper-immediate yang digunakan untuk membuat konstan 32 bit dan menyimpannya pada register rd. Untuk 12 bit terendah akan diisi dengan nol.

6. J-type



Gambar 4-7 Set Instruksi J-type

Instruksi J-type digunakan untuk melakukan unconditional jump. Nilai dari J-immediate adalah signed-offset dari perkalian 2 byte. Offset tersebut dijumlahkan dengan address dari branch instruction untuk mendapatkan target address. Range dari jump adalah +- 1MiB dan akan menyimpan address dari instruksi setelah melakukan jump di register rd.

Masing-masing field-bit yang terdapat di set instruksi memiliki beberapa fungsi tersendiri, yaitu:

1. opcode [6:0], menentukan kelas/format dan keluarga operasinya.
2. rd/rs1/rs2, indeks register tujuan (rd) dan sumber (rs1, rs2)
3. funct3/funct7, sub-opcode untuk membedakan operasi dalam satu opcode
4. immediate, konstanta bertanda, tata letaknya bergantung dengan format.

5. Tentukan tipe instruksi, opcode[6:0], funct3, funct7 (jika ada), dan arti singkat untuk instruksi instruksi RV32I berikut.

SLLI	SUB	NOR	ADDI	XORI	JAL (rd=x0)
SRLI	AND	SLT	SLTI	LUI	JAL
JALR	OR	BEQ	ANDI	LW	ADDI
ADD	XOR	BNE	ORI	SW	SLTIU

Jawab:

Tabel 5-1 Tipe Instruksi, Opcode, Functin, dan Deksripsi dari Instruksi RV32I

Instruksi RV32I	Tipe Instruksi	Opcode[6:0]	funct3	funct7	Deskripsi
SLLI	S-type	0x13	0x1	-	rd = rs1 << imm[0:4]
SUB	R-type	0x33	0x0	0x20	rd = rs1 - rs2
NOR	-	-	-	-	Tidak terdapat instruksi set NOR
ADDI	I-type	0x13	0x0	-	rd = rs1 + imm
XORI	I-type	0x13	0x4	-	rd = rs1 ^ imm
JAL (rd=x0)	J-type	0x6F	-	-	rd = PC+4; PC += imm Pengambilan address pada x0 akan menghasilkan Instruction Access Fault (Exception Code 1).
SRLI	I-type	0x13	0x5	imm[11:5]=0x00	rd = rs1 >> imm[0:4]
AND	R-type	0x33	0x7	0x00	rd = rs1 & rs2
SLT	R-type	0x33	0x2	0x00	rd = (rs1 < rs2)?1:0
SLTI	I-type	0x13	0x2	-	rd = (rs1 < imm)?1:0

	LUI	U-type	0x37	-	-	rd = imm << 12
	JAL	J-type	0x6F	-	-	rd = PC+4; PC += imm
	JALR	J-type	0x67	0x0	-	rd = PC+4; PC = rs1 + imm
	OR	R-type	0x33	0x6	0x00	rd = rs1 | rs2
	BEQ	B-type	0x63	0x0	-	if(rs1 == rs2) PC += imm
	ANDI	I-type	0x13	0x7	-	rd = rs1 & imm
	LW	I-type	0x03	0x2	-	rd = M[rs1+imm][0:31]
	ADDI	I-type	0x13	0x0	-	rd = rs1 + imm
	ADD	R-type	0x33	0x0	0x00	rd = rs1 + rs2
	XOR	R-type	0x33	0x4	0x00	rd = rs1 ^ rs2
	BNE	B-type	0x63	0x1	-	if(rs1 != rs2) PC += imm
	ORI	I-type	0x13	0x6	-	rd = rs1 | imm
	SW	S-type	0x23	0x2	-	M[rs1+imm][0:31] = rs2[0:31]

SLTIU	I-type	0x13	0x3		rd = (rs1 < imm)?1:0

6. **Buat test vector sederhana untuk pengujian Instruction Memory pada Tugas 1 dengan melakukan langkah-langkah berikut.**

```
int main() {
   int i = 1;
   int sum = 0;
   while (i <= 10) {
        sum = sum + i; i = i + 1;
}

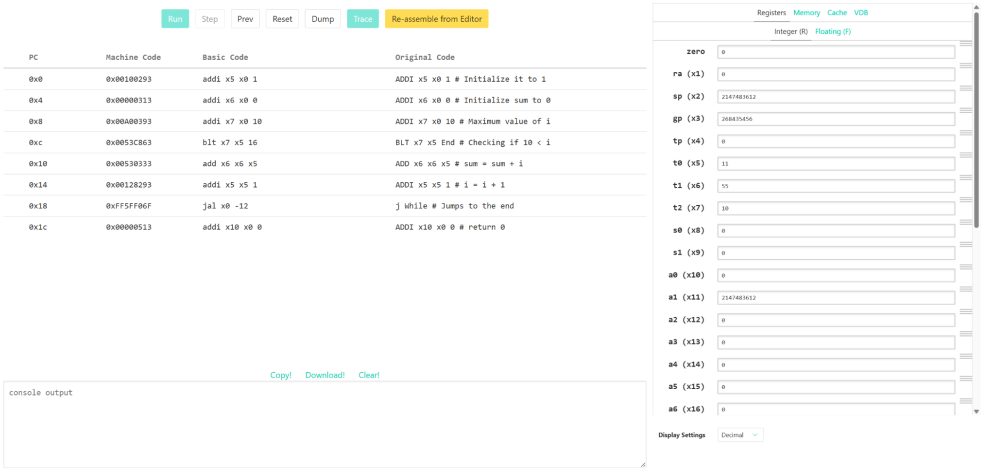
   return 0;
}
```

   a. **Terjemahkan program di atas ke dalam bahasa assembly RISC-V (RV32I). Hanya gunakan instruksi-instruksi ADDI, ADD, dan BLT, kemudian lengkapi dengan label untuk percabangan.**
   Jawab:

   Misalkan nilai dari variabel i dan sum disimpan pada register x5 dan x6.
   Tabel 6-1 Kode Assembly RISC-V dari Kode C untuk Tugas 1

```
# Program : Melakukan While loop hingga mencapai 10 loop (RISC-V RV32I, Venus)
main:
   ADDI x5 x0 1 # Initialize it to 1
   ADDI x6 x0 0 # Initialize sum to 0
   ADDI x7 x0 10 # Maximum value of i
   While:
   BLT x7 x5 End # Checking if 10 < i
   ADD x6 x6 x5 # sum = sum + i
   ADDI x5 x5 1 # i = i + 1
   j While # Jumps to the end
   End:
   ADDI x10 x0 0 # return 0
```

   b. **Lakukan verifikasi di Venus dengan menjalankan program hingga mencapai label akhir, lalu ambil satu screenshot saja yang sekaligus menampilkan panel Text (deretan instruksi) dan panel Registers pada kondisi akhir. Pada screenshot tersebut, tunjukkan dengan jelas bahwa nilai register yang Anda tetapkan sebagai i bernilai 10 dan register yang Anda tetapkan sebagai sum bernilai 55; beri penanda singkat (misalnya keterangan, panah, atau highlight) untuk menyebutkan register mana yang dipakai sebagai i dan sum. Ikuti arahan penghematan kertas dengan hanya menyertakan screenshot pada keadaan akhir eksekusi, tampilan Venus light mode, gambar harus jelas, dan bulak-balik.**
   Jawab:



Gambar 6-1 Hasil Simulasi Venus

Dapat dilihat bahwa pada register x5 yang merupakan variabel i menghasilkan nilai 11 dan pada register x6 yang merupakan variabel sum menghasilkan nilai 55.

c. **Catat hasil machine code (32-bit) dari setiap instruksi yang muncul pada panel Text Venus. (hint: gunakan opsi Dump untuk memberikan semua machine code yang dapat langsung disalin!)**
Jawab:

Tabel 6-2 Machine Code dari Kode C untuk Tugas 1

0x00100293 0x00000313 0x00A00393 0x0053C863 0x00530333 0x00128293 0xFF5FF06F 0x00000513

7. **Buat testbench register file yang: (1) menulis nilai arbitrer bebas di 4 alamat berbeda (mis. x1, x2, x3, dan x4) dengan data yang mudah dicek (misalnya 0x11,0x22,0x33,0x44), satu siklus per penulisan; (2) setelah semua write selesai, membaca 10 alamat (mis. x0 – x9) termasuk 4 alamat tadi. Testbench selfchecking: bandingkan hasil baca dengan expected, laporkan PASS jika semua cocok atau FAIL (alamat, actual, expected) bila tidak. Sesuaikan nama port dan tepi clock stimulus dengan antarmuka serta disiplin baca/tulis register file**
Jawab:

Tabel 7-1 Kode Testbench untuk Register File

```
// Praktikum EL3011 Arsitektur Sistem Komputer
// Modul : 1
// Percobaan : 1
// Tanggal : 21 Oktober 2026
// Nama (NIM) 1 : Danny Rahmat Pratama (13224019)
// Nama (NIM) 2 : Aziiz Adityatama (13224070)
// Nama File : tb_reg_file_rv32i.v
// Deskripsi : Testbench untuk register file RISC-V (RV32I) 32×32
// Timescale for simulation
```

```
`timescale 1ns/1ps

module tb_reg_file_rv32i;

// Variables for testbenches
reg             cu_rdwrite;
reg             clock;
reg     [4:0]   rs1_addr;
reg     [4:0]   rs2_addr;
reg     [4:0]   rd_addr;
reg     [31:0]  rd_in;
wire    [31:0]  rs1;
wire    [31:0]  rs2;

// 32 x Registers (32-bit)
 reg [31:0] rf [0:31];
 integer i;
 integer error;
 reg [31:0] expected[0:9];

// Initialize the clock
always #10 clock = ~clock;

// Taking the main module to the testbench
reg_file_rv32i u0 (
    .clock(clock),
    .cu_rdwrite(cu_rdwrite),
    .rs1_addr(rs1_addr),
    .rs2_addr(rs2_addr),
    .rd_addr(rd_addr),
    .rd_in(rd_in),
    .rs1(rs1),
    .rs2(rs2)
);

task init;
begin
    for (i = 0; i < 32; i = i + 1) begin
        rf[i] = 32'b0;
    end
    cu_rdwrite <= 0;
    clock <= 0;
    rs1_addr <= 0;
    rs2_addr <= 0;
    rd_addr <= 0;
    rd_in <= 0;
    error = 0;
    for (i = 0; i < 10; i = i + 1) begin
        expected[i] = 32'b0;
```

```
    end
    expected[1] = 32'h00000011;
    expected[2] = 32'h00000022;
    expected[3] = 32'h00000033;
    expected[4] = 32'h00000044;
end
endtask

task testing;
begin
    @(negedge clock);
    cu_rdwrite <= 1;

    @(negedge clock);
    rd_addr <= 0;
    rd_in <= 32'hFFFFFFFF;

    for(i = 1; i < 5; i = i + 1) begin
        @(negedge clock);
        rd_addr <= i;
        rd_in <= expected[i];
    end

    @(negedge clock);
    cu_rdwrite <= 0;

    for(i = 0; i < 10; i = i + 1) begin
        @(negedge clock);
        rs1_addr <= i;
        @(negedge clock);
        #1;
        if(rs1 !== expected[i]) begin
            $display("-----------------------------------");
            $display("FAIL");
            $display("T = %0t | alamat  = %h | actual = %h | expected
= %h", $time, rs1_addr, rs1, expected[i]);
            error = error + 1;
        end else begin
            $display("-----------------------------------");
            $display("PASS");
        end
    end
end
endtask

initial begin
    $dumpfile("reg_file_rv32i.vcd");
```

```
    $dumpvars(0, tb_reg_file_rv32i);

    init;
    testing;

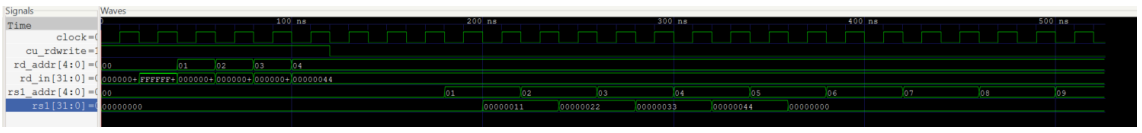
    #5
    $display("Testbench selesai dengan %d error", error);
    $finish;

end

endmodule
```

Tabel 7-2 Hasil Testbench untuk Register File

```
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
----------------------------------
PASS
Testbench selesai dengan          0 error
```



Gambar 7-1 Hasil Waveform dari Testbench

8. **Pilih 5 instruksi pada Tugas Pendahuluan nomor 5. Terjemahkan ke machine code heksadesimal, kemudian tuliskan opcode, funct7, funct3, dan trimmed_instr (instr[31:7]). Ini akan digunakan untuk validasi tugas 3 (CU).**

   Jawab:

Tabel 8-1 Instruksi Set untuk Control Unit

Instruction Set	Hexadecimal	Opcode	funct7	funct3	trimmed_instr [31:7]

ADD x5 x5 x6	0x006282B3	0110011	0000000	000	00000000 01100010 100000101
SRLI x6 x5 5	0x0052D313	0010011	-	101	00000000 01010010 110100110
LW x6 12(x2)	0x00C12303	0000011	-	010	00000000 11000001 001000110
SW x5 8(x2)	0x00512423	0100011	-	010	00000000 01010001 001001000
BNE x5 x6 4	0x00629263	1100011	-	001	00000000 01100010 100100100

9. **Tentukan seluruh keluaran CU (16 sinyal) untuk input instruksi yang kalian pilih pada TP nomor 5. Buat tabel dengan instruksi per baris dan sinyal kontrol per kolom sebagai expected output pengujian Tugas 3.**
   Jawab:

   Tabel 9-1 Keluaran CU dari Instruksi Set Terpilih

Instruksi Set	cu_ALU1src	cu_ALU2src	cu_immtype	cu_ALUtype	cu_adtype	cu_gatype	cu_shiftype	cu_slttype	cu_rdtype	cu_rdwrite	cu_loadtype	cu_store	cu_storetype	cu_branch	cu_bbranchtype	cu_jump
ADD x5 x5 x6	0	0	–	00	0	–	–	-	00	1	–	0	–	0	–	0
SRLI x6 x5 5	0	1	000	10	-	–	01	-	00	1	—	0	–	0	–	0
LW x6 12(x2)	0	1	000	00	0	–	–	-	01	1	010	0	–	0	—	0
SW x5 8(x2)	0	1	001	00	0	–	–	-	–	0	—	1	10	0	–	0
BNE x5 x6 4	1	1	010	00	0	–	–	-	–	0	–	0	–	1	010	0

Referensi:

[1] Cornell University, "06-cpu-notes-bw," CS 3410, Spring 2019. [Online]. Available: https://www.cs.cornell.edu/courses/cs3410/2019sp/schedule/slides/06-cpu-notes-bw.pdf. [Accessed: Oct. 27, 2026].

[2] RISC-V International, "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA," [Online]. Available: https://docs.riscv.org/reference/isa/_attachments/riscv-unprivileged.pdf. [Accessed: Oct. 27, 2026].
[3] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition*. [Online]. Available: https://github.com/tanglang96/awesome-books/blob/master/books/Computer-Organization-and-Design-The-Hardware-Software-Interface-RISC-V-Edition.pdf. [Accessed: Oct. 27, 2026].
[4] Program Studi Teknik Elektro, *Modul Praktikum EL3011 Arsitektur Sistem Komputer*. Bandung: Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung, 2026. Local file: Modul_EL3011_Arsikom_2026.pdf.